University of Science and Technology of Oran Mohamed Boudiaf
Faculty of Mathematics and Informatics
Department of Computer Science

Version control

Why history matters, and how to use Git every day

Mohammed Said Belaid
Eng 5 SECI, DevOps

Opening

Today's agenda

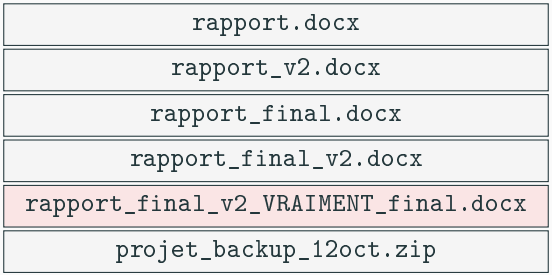
1. Why should we care?
2. Life without version control
3. A short history: each generation fixes the previous one
4. Everyday Git
5. Branches and merges
6. Working as a team
7. Wrap-up

> Today's question: *why* is a reliable, shared history the foundation of everything else in DevOps?

Sound familiar?

```
rapport.docx
rapport_v2.docx
rapport_final.docx
rapport_final_v2.docx
rapport_final_v2_VRAIMENT_final.docx
projet_backup_12oct.zip
```

Sound familiar?



- Which one is the latest? Which one did you send to the teacher?
- What changed between `v2` and `final`? Who changed it? Why?
- Your partner edited `rapport_final.docx` at the same time as you...

Production is broken

"But it worked last week!"

1. What changed since last week?
2. Who changed it?
3. Why was it changed?
4. Can we go back to last week's version, now, safely?

Production is broken

"But it worked last week!"

1. What changed since last week?
2. Who changed it?
3. Why was it changed?
4. Can we go back to last week's version, now, safely?

> Without a reliable history, none of these questions has an answer, and lecture 1's flow (small, frequent changes) and feedback (find what broke, fast) are impossible.

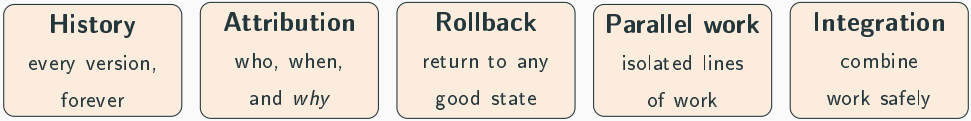
Life without version control

Five pains

Pain	Consequence
Lost work	A disk crash, a bad copy-paste: days of work are gone
Overwriting each other	Last one to save wins; the other's work silently vanishes
No "why"	The code says *what*, never *why* it became like this
No way back	A bad change in production; nobody knows the last good state
No parallel work	"Don't touch the project, I'm working on it"

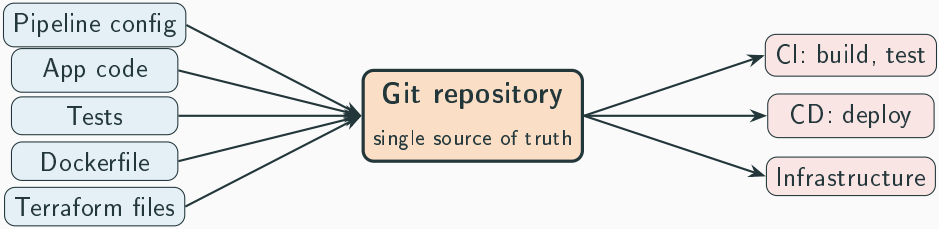
Each pain becomes a requirement for a tool.

What we need from a tool



A version control system (VCS) is a time machine (history, attribution, rollback) plus a collaboration protocol (parallel work, integration).

Version control in DevOps: everything as code



Everything that builds, tests, deploys or runs the system lives in version control: every later session starts from a commit.

> DORA (*Accelerate*, 2018): "use version control for *all* production artifacts" is one of the capabilities that predict delivery performance.

What should (not) be in version control?

Think for a minute: what goes in, what stays out?

What should (not) be in version control?

Think for a minute: what goes in, what stays out?

In

- Source code and tests
- Build and pipeline configuration
- Infrastructure definitions, scripts
- Documentation

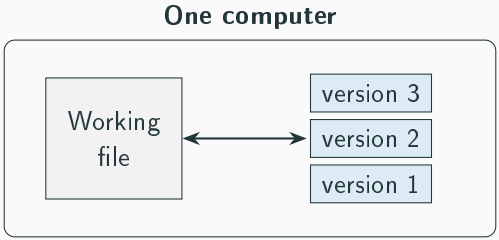
Out

- Secrets: passwords, API keys
- Build outputs (`.class`, `.o`, binaries)
- Downloaded dependencies (`node_modules/`)
- Personal IDE settings

Rule: version what you *cannot regenerate*. And a secret pushed once stays in the history forever. ($\rightarrow$ `.gitignore`)

A short history

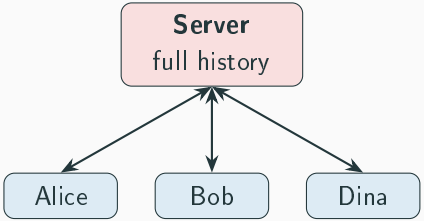
Generation 1: local version control



- RCS (1982): keeps the versions of each file on your machine
- Solves: history, rollback, but only for one person

> New problem: how do several people share their work?

Generation 2: centralised version control



- CVS (1990), Subversion/SVN (2000)
- Solves: team work, one shared history
- New problems:
  - server down → nobody can commit
  - network needed for every operation (log, diff, commit)
  - branches = copies of the whole tree: slow, rarely used
  - often: lock a file before editing it

> A single point of failure, and branching so costly that people avoid it.

Generation 3: distributed, the birth of Git (2005)

- 2002-2005: the Linux kernel uses BitKeeper, a proprietary distributed VCS, under a free licence
- April 2005: the licence is withdrawn after a dispute
- Linus Torvalds writes his own tool: Git manages its own source code within days; Linux 2.6.12 is released with it in June

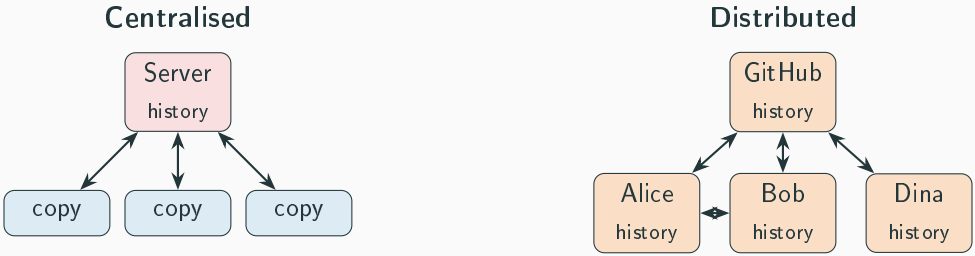
Generation 3: distributed, the birth of Git (2005)

- 2002-2005: the Linux kernel uses BitKeeper, a proprietary distributed VCS, under a free licence
- April 2005: the licence is withdrawn after a dispute
- Linus Torvalds writes his own tool: Git manages its own source code within days; Linux 2.6.12 is released with it in June

Design goals:

- Speed: almost every operation is local
- Fully distributed: no mandatory central server
- Non-linear development: thousands of parallel branches
- Integrity: corruption or tampering must be detected
- Scale to a huge project (the kernel: thousands of contributors)

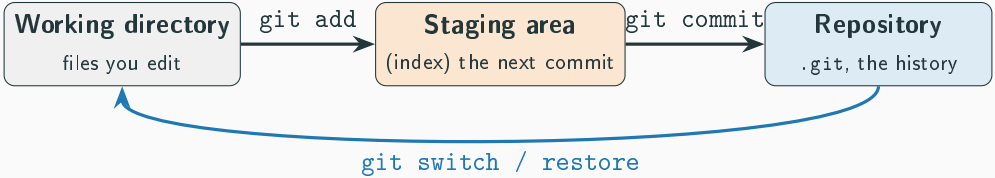
Centralised vs distributed



- Every clone is a full repository: complete history, works offline, is a backup
- Commit, log, diff, branch: local and instant; share only when ready (`push`, `pull`)
- "The central repository" (GitHub) is a team convention, not a technical need

Everyday Git

The three areas: why a staging area?



- You changed 5 files: a bug fix, a new feature, and a debug `print`
- The staging area lets you choose what goes into the next commit
- → one commit = one logical change, not "everything I did today"

> Why it matters: a commit that does one thing can be reviewed, understood, and undone on its own.

The daily loop

```
$ git status              # what changed? what is staged?
$ git diff                # my changes, line by line
$ git add cart.py         # choose what goes into the next commit
$ git diff --staged       # review what I am about to commit
$ git commit -m "Fix crash when the cart is empty"
$ git log --oneline       # check the result
a158e2f Fix crash when the cart is empty
c337f7a Add checkout page
```

- `git status` is your compass: run it before and after every command, and read it (it often tells you the next command)
- Commit when one logical change is done: several times a day, not once a week

> `git diff --staged` before each commit is your own first code review.

Reading the history

```
$ git log --oneline --graph --all     # the whole picture
$ git show a158e2f                    # one commit: author, message, diff
$ git log -p -- cart.py               # every change ever made to one file
$ git diff main feature               # what differs between two branches
$ git blame cart.py                   # who last changed each line
```

- Each commit has a unique ID (`a158e2f`), identical on every clone
- It names an exact version of the whole project: "`a158e2f` passed the tests", "`a158e2f` runs in production". This is how CI and CD will talk (remember Knight Capital)

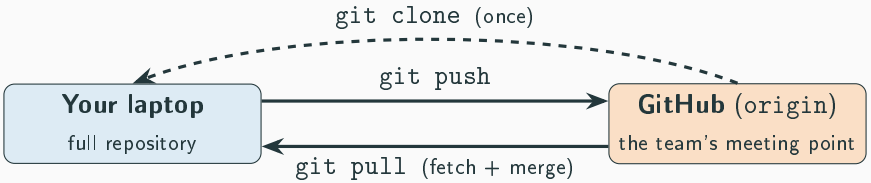
> History is only useful if you can read it: small commits and good messages make these commands answer real questions.

Undoing things

Situation	Command	After a push?
I broke a file, I want it back	`git restore f`	n/a (edits are lost!)
I staged a file by mistake	`git restore --staged f`	n/a
Fix the last commit (message, file)	`git commit --amend`	no: rewrites it
Undo my last local commit	`git reset HEAD~1`	no: rewrites history
A shared commit broke production	`git revert <id>`	yes: adds a commit

Rule: before `push`, you may rewrite *your* history; after `push`, only add to it (`revert`). Safety net: `git reflog`.

Working with a remote



- `pull`: get the others' commits, before you start and before you push
- `push`: publish your commits; rejected if GitHub has commits you lack
- `origin/main`: your laptop's last view of GitHub's `main`

> Daily rhythm: `pull` → branch → small commits → `pull` → `push`. Never `push --force` on a shared branch.

Keeping the repository clean: `.gitignore`

```
# .gitignore
.env
*.log
build/
__pycache__/
node_modules/
.vscode/
```

- One pattern per line; commit the `.gitignore` itself, so the whole team shares it
- `git status` stays readable: only what matters shows up
- Ready-made templates per language: `github.com/github/gitignore`
- Committed by mistake? `git rm --cached .env` stops tracking it (the file stays on disk)

> `.gitignore` does not erase a secret already committed: it stays in the history. Once pushed, change the password.

Branches and merges

Why branches?

You are half-way through a new feature.
An urgent bug is reported in production. What do you do?

Why branches?

You are half-way through a new feature.
An urgent bug is reported in production. What do you do?

- Commit the half-done feature? → you ship broken code
- Copy the project folder? → two diverging copies, merged by hand
- Throw your work away? → no

Why branches?

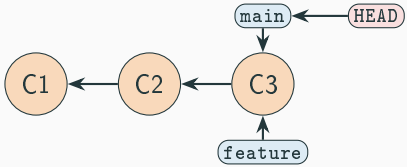
You are half-way through a new feature.
An urgent bug is reported in production. What do you do?

- Commit the half-done feature? → you ship broken code
- Copy the project folder? → two diverging copies, merged by hand
- Throw your work away? → no

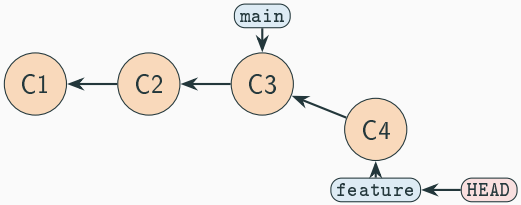
A branch is an isolated line of work: a feature, a fix, an experiment. It does not disturb `main` or your colleagues.

A branch is just a pointer

`git branch feature`

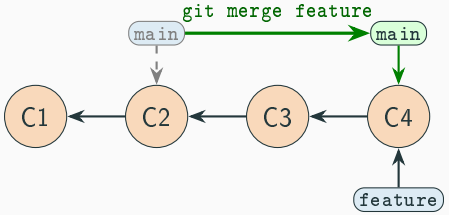


`git switch feature;` **commit**



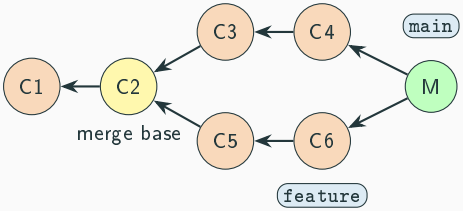
- A branch = a name pointing to one commit. It moves forward at each commit
- `HEAD` = "where am I?": the branch you are currently on
- Nothing is copied: creating a branch is instant → branch for every task

Fast-forward merge



- `main` has not moved since `feature` was created: no divergence
- Git simply moves the pointer forward. No new commit, nothing to combine

Three-way merge



Git compares the two tips and their common ancestor, which tells *who* changed what:

Base	main	feature	Result
A	A	B	B (only feature changed it)
A	B	A	B (only main changed it)
A	B	C	conflict: both changed it

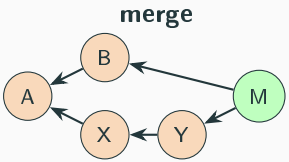
Conflicts are a feature, not a bug

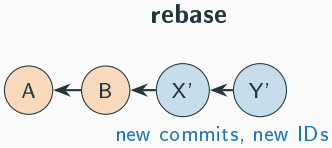
```
<<<<<<< HEAD
TAX_RATE = 0.19
=======
TAX_RATE = 0.09
>>>>>>> feature
```

- Two people changed the same lines in different ways since the merge base
- Git refuses to guess: which value is right is a business decision, not a text operation

> A tool that "always takes the most recent version" would silently ship a wrong tax rate. A conflict is a conversation between two developers, and the longer you wait to merge, the bigger it gets.

Merge or rebase?





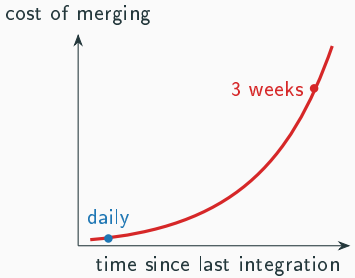
	merge	rebase
History	true: shows what really happened	rewritten: linear, easier to read
Commits	kept as they are	copied on top of the target: new IDs

Golden rule: never rebase commits that others already have. (Why? Rebased commits are *new* commits: your colleagues' copies no longer match.)

Working as a team

Integration hell

- Two developers on long-lived branches for 3 weeks
- Every day, the branches diverge more
- Merge day: dozens of conflicts, broken tests, nobody remembers the reasons
- Sounds familiar? Lecture 1: big, rare releases, the vicious circle



> Same lesson as lecture 1, at the scale of code: integrate small and often. If it hurts, do it more often.

Branching strategies

Strategy	Idea	Fits
Git Flow (2010)	`main`, `develop`, feature, release and hotfix branches; long-lived	Versioned releases (desktop, embedded)
GitHub Flow (2011)	`main` always deployable; short feature branch + pull request	Web apps, continuous deployment
Trunk-based	Everyone integrates into `main` at least daily; branches live hours	High-performing DevOps teams

- DORA (*Accelerate*): trunk-based development (few active branches, branches living less than a day, no code freeze) predicts higher delivery performance

> In this course: GitHub Flow (short branches + pull requests, next session).

Good commits

Bad

```
fix
update
wip
changes from yesterday
```

Good

```
Fix crash when the cart is empty

The total was computed before checking
that the cart had items, which caused a
division by zero on the checkout page.
```

- Small and atomic: one logical change, which builds and passes the tests
- Subject: short, imperative ("Fix", "Add"); body: explains *why*, not what (the diff shows what)

> In 6 months, the commit message is the only place where the "why" survives.

Wrap-up

Back to the broken production

Question	Git answer
What changed since last week?	`git log --since“1 week ago”, =git diff`
Who changed this line, and why?	`git blame`, then read the commit message
Which commit broke it?	`git bisect`: binary search in the history
Can we go back, now, safely?	`git revert` (a new commit that undoes one change)

> Each answer works only if commits are small, have a good message, and everything is versioned.

Key takeaways

1. Version control = time machine + collaboration protocol; everything that builds and runs the system is versioned
2. Each generation of VCS fixed the problems of the previous one; Git: distributed, fast, trustworthy
3. Daily loop: `status`, `add`, `commit`, `pull`, `push`; the staging area makes each commit one logical change
4. Undo safely: rewrite only what is not pushed; once shared, `revert`. A commit ID names an exact version
5. Branches are cheap pointers; conflicts are Git refusing to guess
6. Integrate small and often: long-lived branches recreate the vicious circle

Today's lab and further reading

Lab 1

- A small Python app: feature branch, hotfix, conflict, rebase, undo
- Push it to your pair's GitHub repository (kept all semester)

Reading

- S. Chacon, B. Straub, *Pro Git*, 2nd ed., Apress (free online: git-scm.com/book), ch. 1-3
- M. Fowler, "Patterns for Managing Source Code Branches", martinfowler.com, 2020
- C. Beams, "How to Write a Git Commit Message", cbea.ms, 2014

Next session

Collaboration and quality

Why review code? Why write tests?
How do pull requests turn a branch into a team decision?

Questions?